

Database Maintenance

In this chapter, you'll learn how to perform common database maintenance tasks.

Backing Up Data

Like all other data, MySQL data must be backed up regularly. Because MySQL databases are disk-based files, normal backup systems and routines can back up MySQL data. However, those files are always open and in use, so normal file copy backups might not always work.

Here are possible solutions to this problem:

- Use the command-line utility `mysqldump` to dump all database contents to an external file. This utility should ideally be run before regular backups occur so the dumped file will be backed up properly.
- Use the command-line utility `mysqlhotcopy` to copy all data from a database. (This utility is not supported by all database engines.)
- Use MySQL to dump all data to an external file using `BACKUP TABLE` or `SELECT INTO OUTFILE`. Both statements take the name of a system file to be created; and that file must not already exist, or an error will be generated. Data can be restored by using `RESTORE TABLE`.

Tip

Flush Unwritten Data First To ensure that all data is written to disk (including any index data), you might need to use a `FLUSH TABLES` statement before performing your backup.

Performing Database Maintenance

MySQL features a series of statements that can (and should) be used to ensure that databases are correct and functioning properly.

Here are some statements you should be aware of:

- `ANALYZE TABLE` is used to check that table keys are correct. `ANALYZE TABLE` returns status information, as shown here:

► Input

```
| ANALYZE TABLE orders;
```

► Output

Table	Op	Msg_type	Msg_text
crashcourse.orders	analyze	status	OK

- `CHECK TABLE` is used to check tables for a variety of problems. Indexes are also checked on a MyISAM table. `CHECK TABLE` supports a series of modes for use with MyISAM tables. `CHANGED` checks tables that have changed since the last check, `EXTENDED` performs the most thorough check, `FAST` checks only tables that were not closed properly, `MEDIUM` checks all deleted links and performs key verification, and `QUICK` performs a quick scan only. In this example, `CHECK TABLE` finds and repairs a problem:

► Input

```
| USE crashcourse;  
| CHECK TABLE orders, orderitems;
```

► Output

Table	Op	Msg_type	Msg_text
crashcourse.orders	check	status	OK
crashcourse.orderitems	check	warning	Table is marked as crashed
crashcourse.orderitems	check	status	OK

- If MyISAM table access produces incorrect and inconsistent results, you might need to repair the table by using `REPAIR TABLE`. This statement should not be used frequently, and if regular use is required, there is likely a far bigger problem that needs to be addressed.
- If you delete large amounts of data from a table, you should use `OPTIMIZE TABLE` to reclaim previously used space and optimize the performance of the table.

Diagnosing Startup Problems

Server startup problems usually occur when a change has been made to the MySQL configuration or to the server. MySQL reports errors when startup problems occur, but because most MySQL servers are started automatically as system processes or services, these messages might not be seen.

When troubleshooting system startup problems, try to manually start the server first. You start the MySQL server by executing `mysqld` on the command line. Here are several important command-line options for `mysqld`:

- `--help` displays help as a list of options.
- `--safe-mode` loads the server minus some optimizations.
- `--verbose` displays full text messages. You can use it in conjunction with `--help` for more detailed help messages.
- `--version` displays version information and then quits.

Several additional command-line options (pertaining to the use of log files) are listed in the next section.

Reviewing Log Files

MySQL maintains a series of log files that administrators rely on extensively. These are the primary log files:

- **Error log:** This log contains details about startup and shutdown problems and any critical errors. It is usually named `hostname.err` and located in the `data` directory. You can change this name by using the `--log-error` command-line option.
- **Query log:** This log details all MySQL activity and can be very useful in diagnosing problems. This log file can get very large very quickly, so it should not be used for extended periods of time. It is usually named `hostname.log` and located in the `data` directory. You can change this name by using the `--log` command-line option.
- **Binary log:** This log logs all statements that updated (or could have updated) data. It is usually named `hostname-bin` and located in the `data` directory. You can change this name by using the `--log-bin` command-line option. Note that this log file was added in MySQL 5; the update log is used in earlier versions of MySQL.
- **Slow query log:** As its name suggests, this log details any queries that execute slowly. This log can be useful in determining where database optimizations are needed. It is usually named `hostname-slow.log` and located in the `data` directory. You can change this name by using the `--log-slow-queries` command-line option.

When logging is being done, you can use the `FLUSH LOGS` statement to flush and restart all log files.

Tip

Use MySQL Workbench for DBMS Maintenance Throughout this book, we've used MySQL Workbench primarily to write and execute SQL statements. But MySQL Workbench has evolved into more than just a SQL editor. In fact, the Server menu and the Administration tab of the Navigator provides interactive access to all of the commands listed above—and more.

Summary

In this chapter, you learned about some basic MySQL database maintenance tools and techniques.